

DocProcessor

Dokumentation in eine überprüfbare Feature-Map für die QA-Automatisierung umwandeln

Zusammenfassung

DocProcessor ist ein eigenständiges, vollständig entkoppeltes Go-Modul, das Projektdokumentationen lädt, strukturierte Feature-Maps erstellt und die Verifizierungsabdeckung nachverfolgt. Es ist darauf ausgelegt, mit LLM-Agenten für intelligente Feature-Extraktion zusammenzuarbeiten, enthält jedoch auch heuristische Extraktionsmethoden für den Offline-Betrieb.

Kurzbeschreibung

Ein projektunabhängiges Go-Modul zur Dokumentationsverarbeitung und Feature-Map-Extraktion. Es parst Dokumentationen in strukturierte Feature-Maps und verfolgt, welche Features verifiziert sind – entweder mithilfe von LLM-Agenten für intelligente Extraktion oder heuristischen Methoden im Offline-Modus – und versorgt die QA-Automatisierung mit einer Anti-Bluff-Garantie, die stets der Realität entspricht.

Ausführliche Beschreibung

Jedes Softwareteam lebt mit derselben stillen Lüge: Die Dokumentation verspricht Features, die Tests decken etwas Ähnliches ab, und niemand kann mit Sicherheit sagen, ob beide dasselbe Produkt beschreiben. DocProcessor wurde entwickelt, um diese Lücke sichtbar und messbar zu machen. Ausgehend von der Projektdokumentation erstellt es eine strukturierte Feature-Map – ein enumeriertes, maschinenlesbares Modell aller Funktionen, die das Produkt laut Dokumentation bieten soll – und verfolgt die Verifizierungsabdeckung im Vergleich dazu. So wird aus der Frage „*Wird dieses dokumentierte Feature tatsächlich nachgewiesen?*“ keine Flur-Diskussion mehr, sondern eine Abfrage mit klarer Antwort. Das Modul arbeitet bewusst in zwei

Modi: Es nutzt LLM-Agenten für intelligente, semantische Feature-Extraktion, falls verfügbar, und greift ansonsten auf einen heuristischen Parser für den vollständig offlinefähigen Betrieb zurück. Dadurch ist es nie von einem Modell abhängig und läuft identisch in einer isolierten CI-Umgebung oder auf dem Laptop eines Entwicklers im Flugmodus.

Architektonisch handelt es sich um ein eigenständiges, projektunabhängiges, vollständig entkoppeltes Go-Modul (CONST-051(B)): Es enthält keinerlei projektspezifische Werte und wird von Anwendern als gleichberechtigtes Code-Submodul eingebunden, sodass jedes Projekt es übernehmen kann, ohne fremde Annahmen zu erben. Zudem hält es sich selbst an die Standards, die es von anderen einfordert – seine eigenen Aussagen unterliegen dem Anti-Bluff-Prinzip (CONST-035) und den Regeln zur vollständigen Automatisierungsabdeckung (CONST-048). Das bedeutet, dass jede im README beworbene Funktion durch einen automatisierten Test oder ein Challenge-Skript überprüft wird, das reales, nutzerrelevantes Verhalten bestätigt – und nicht nur mit Exit-Code 0 endet. Nutzerrelevante Strings durchlaufen die i18n-Übersetzungsschnittstelle CONST-046. Ziel all dessen ist ein geschlossener Kreislauf: DocProcessor bildet die Eingangsseite des QA-Zyklus, den HelixQA schließt – es extrahiert die Feature-Map aus der Dokumentation, HelixQA belegt jedes gemappte Feature mit erfassten Laufzeitnachweisen, und Dokumentation, Tests und ausgeliefertes Verhalten werden gezwungen, sich anzugleichen, statt sich Release für Release unbemerkt auseinanderzuentwickeln.

Warum wir es entwickelt haben

Dokumentation und Tests driften auseinander: Die Dokumentation verspricht Features, die kein Test nachweist, und die QA kann nicht ohne Weiteres bestimmen, was „vollständig“ bedeutet. DocProcessor verwandelt Dokumentationen in maschinenlesbare Feature-Maps, sodass die Verifizierungsabdeckung an dem gemessen werden kann, was tatsächlich versprochen wurde.

Warum es ein Game-Changer ist

Es verwandelt die unschärfste Frage in der Softwareauslieferung – *„Entspricht das, was wir ausgeliefert haben, dem, was wir auszuliefern behaupten?“* – in etwas Automatisierbares und kontinuierlich Überprüfbares. Und das ohne harte AI-Abhängigkeit: LLM-Extraktion, wenn ein Modell verfügbar ist,

Heuristiken, wenn nicht – sodass dieselbe Garantie in jeder Umgebung gilt, vom Offline-Runner bis zur vollautomatisierten Pipeline.

Was innovativ ist

- Extraktion einer Dokumentation-Feature-Map mit Verifizierungsabdeckungs-Tracking.
- Doppelte Extraktion: LLM-Agent-gesteuert oder heuristisch/offline.
- Projektunabhängige, konfigurationsfreie Entkopplung (CONST-051(B)).
- Anti-Bluff-Selbstverifizierung: README-Aussagen werden durch Tests/Challenges abgesichert (CONST-035/048).

Herausforderungen & Lösungen

- **Modellunabhängiger Betrieb:** gelöst durch einen heuristischen Extraktions-Fallback, sodass das Modul offline funktioniert.
- **Dokumentation und Realität im Einklang halten:** gelöst durch strukturierte Feature-Maps und Verifizierungsabdeckungs-Tracking, integriert in den QA-Prozess.
- **Wiederverwendbarkeit:** gelöst durch strikte Entkopplung und gleichberechtigten Submodul-Einsatz im Codebase.
- **Glaubwürdigkeit der eigenen Aussagen:** gelöst durch Anti-Bluff-Tests/Challenges für jede beworbene Funktion.

Technologie-Stack (Warum + Wie)

- **Go (1.25+)** – Modulkern; Lizenz: Apache-2.0.
- **LLM-Agenten** – intelligente semantische Feature-Extraktion (optional).
- **Heuristischer Parser** – Offline-Fallback für Feature-Extraktion.
- **i18n-Übersetzer (pkg/i18n)** – CONST-046 lokalisierte Strings.
- **Challenge-Harness** – Anti-Bluff-Verifizierung der Modulaussagen.